



COLLEGE ONLINE STOCK MANAGEMENT SYSTEM



PROJECT REPORT

Submitted

by

SARANYA M

24133051802522037

RUTHRA P

24133051802522036

INDHUMATHI R

24133051802522015

in partial fulfilment for the award of the degree of

BACHELOR OF COMPUTER SCIENCE

**IMMACULATE COLLEGE FOR WOMEN,
VIRIYUR**

DEPARTMENT OF COMPUTER SCIENCE

ANNAMALAI UNIVERSITY

Oct 2026

ACKNOWLEDGEMENT

We thank Almighty God for giving us the strength and the opportunity to complete this project successfully.

We would like to extend our gratitude to our Secretary, Rev. Mo. FINNIAN MARY KURUSAMMAL, FHIM, for her continuous efforts towards our education.

We are greatly indebted to the Head of the Department, Mr. A. GEORGE AROCKIA RAJ, HOD, for his constant support and encouragement in completing this project.

We express our sincere thanks to our project guide, Mr. A. GEORGE AROCKIA RAJ, HOD, who patiently helped us to understand the project and supported us throughout the work.

We also thank our project co-ordinator and all the faculty members of the Department of Computer Science for their kind support, suggestions and encouragement.

We thank the members of our family for their support and blessings. Last but not least, we thank our friends and classmates for their constant support in developing this project.

BY

SARANYA M

RUTHRA P

INDHUMATHI R

TABLE OF CONTENT

ABSTRACT	1
1. INTRODUCTION	2
2. OBJECTIVES AND SCOPE	3
2.1 OBJECTIVES OF COLLEGE ONLINE STOCK MANAGEMENT SYSTEM	3
2.2 SCOPE OF COLLEGE ONLINE STOCK MANAGEMENT SYSTEM	3
2.3 BENEFITS	3
2.4 OVERVIEW	4
3. FEASIBILITY STUDY	5
3.1 ECONOMIC FEASIBILITY	5
3.2 TECHNICAL FEASIBILITY	5
3.3 BEHAVIOURAL FEASIBILITY	5
4. SYSTEM STUDY	6
4.1 EXISTING SYSTEM	6
4.2 PROPOSED SYSTEM	6
5. TESTING AND IMPLEMENTATION	7
5.1 TESTING OBJECTIVE	7
5.2 TESTING PRINCIPLE	7
5.3 WHITE-BOX TESTING	7
5.4 BLACK-BOX TESTING	7
5.5 TESTING STRATEGIES	7
5.6 TESTING FUNDAMENTALS	7
5.7 TESTING INFORMATION FLOW	8
5.8 UNIT TESTING	8
5.9 INTEGRATION TESTING	8
5.10 SYSTEM TESTING	8
5.11 ACCEPTANCE TESTING	8
5.12 TEST CASES	8
5.13 IMPLEMENTATION	9
5.13.1 END USER TRAINING	9
5.13.2 END USER EDUCATION	9
5.13.3 TRAINING OF APPLICATION SOFTWARE	9
5.13.4 POST IMPLEMENTATION VIEW	9
6. SYSTEM CONFIGURATION	10
6.1 HARDWARE REQUIREMENTS	10
6.2 SOFTWARE REQUIREMENTS	10
7. COLLEGE ONLINE STOCK MANAGEMENT SYSTEM DATA FLOW DIAGRAM	11
7.1 0 LEVEL DFD FOR COLLEGE ONLINE STOCK MANAGEMENT SYSTEM	11
7.2 LEVEL 1 DFD FOR COLLEGE ONLINE STOCK MANAGEMENT SYSTEM	11
7.3 LEVEL 2 DFD FOR COLLEGE ONLINE STOCK MANAGEMENT SYSTEM	12
8. MODULES OF COLLEGE ONLINE STOCK MANAGEMENT SYSTEM	13
8.1 USER REGISTRATION	13
8.2 LOGIN MODULE	13
8.3 ADMIN MODULE	13
8.4 STOCK MANAGEMENT MODULE	13
8.5 STAFF MODULE	13
8.6 STUDENT MODULE	13
9. TECHNOLOGY USED	14
9.1 FRONT END TECHNOLOGY	14
9.2 BACK END TECHNOLOGY	14
10. CODING	16
11. SCREEN SHOTS	30
12. CONCLUSION	32
13. REFERENCE	33

ABSTRACT

The College Online Stock Management System is a web-based application designed to record, monitor and control the stock of the physical resources of a college, such as chairs, tables, benches, computers and laboratory equipment. In many colleges the details of these resources are still kept in handwritten registers, which makes searching, updating and verifying the stock slow and error-prone.

The proposed system replaces the registers with a central database that is accessed through a web browser. It has three types of users. The admin adds, updates and deletes items and approves or rejects requests. Staff members and students log in to view the available items and to send requests for the items they need. When a request is approved, the available quantity is updated automatically, so the stock figures are always correct.

The registration module is used to create new accounts, and the login module is used to enter the portal according to the role of the user. The system is developed using React for the front end, Node.js with the Express framework for the server-side processing and MySQL for the database. It reduces paper work, saves time, avoids counting mistakes and helps the college management to plan its resources.

CHAPTER 1

1. INTRODUCTION

Every college owns a large number of physical resources such as chairs, tables, benches, computers, projectors and laboratory equipment. These resources are used every day by students and staff, and they must be counted, allotted, repaired and replaced regularly. In most colleges this work is still done with handwritten stock registers, which are difficult to search, easy to lose and often out of date.

The College Online Stock Management System is a web-based application that keeps the details of all these resources in one central database. The administrator can add new items, update the quantity and condition of existing items, and remove items that are no longer in use. Staff members and students can log in to view the items that are available and can send a request for the items they need. The administrator approves or rejects each request, and the available quantity is updated automatically.

The system is built with React for the user interface, Node.js (with the Express framework) for the server-side logic and MySQL for storing the data. It runs on any computer with a web browser, and Node.js and MySQL are the only tools needed on the computer that hosts the server, so no costly software is required. Separate login accounts for the admin, staff and students make sure that every person sees only the pages that belong to his or her role.

By replacing paper registers with an online system, the college can know at any moment how many items it owns, how many are in use, how many are available and which items are damaged. This saves time, reduces mistakes and helps the management to plan new purchases with correct figures.

CHAPTER 2

2. OBJECTIVES AND SCOPE

2.1 OBJECTIVES OF COLLEGE ONLINE STOCK MANAGEMENT SYSTEM

The main aim of the project is to build a simple, reliable and secure system that manages the stock of a college through the web. The specific objectives are:

- To maintain a complete and accurate record of furniture (chairs, tables, benches) and laboratory equipment.
- To provide separate secure logins for the admin, staff and students.
- To allow the admin to add, update and delete stock items at any time.
- To let staff and students view the available items and send requests online.
- To update the available quantity automatically when a request is approved.
- To record damaged or repair-needed items so that they can be attended to quickly.
- To generate quick stock reports and highlight items that are running low.

2.2 SCOPE OF COLLEGE ONLINE STOCK MANAGEMENT SYSTEM

The system covers the stock of a single college or department. It handles two main categories of items, namely furniture and laboratory equipment, and more categories can be added easily. It supports three types of users: the administrator, the staff and the students. The project can be extended later to include purchase records, barcode scanning, email alerts and multiple campuses.

2.3 BENEFITS

- Paper work is reduced and the stock details are always available online.
- Searching for an item or checking its quantity takes only a few seconds.
- Manual counting mistakes are avoided because the quantity is updated by the system.
- The admin knows which items are low in stock, damaged or in demand.
- Passwords are stored in encrypted form, so the data is protected.
- The system is low-cost because it uses free and open-source software.

2.4 OVERVIEW

This report is divided into thirteen chapters. Chapter 1 introduces the project. Chapter 2 explains the objectives, scope and benefits. Chapter 3 discusses the feasibility of the system, and Chapter 4 compares the existing and the proposed system. Chapter 5 describes testing and implementation, and Chapter 6 lists the hardware and software needed. Chapter 7 presents the data flow diagrams, Chapter 8 explains the modules, and Chapter 9 describes the technologies used. Chapters 10 and 11 contain the coding and the screenshots. Chapter 12 gives the conclusion and Chapter 13 lists the references.

CHAPTER 3

3. FEASIBILITY STUDY

A feasibility study is carried out to find out whether the proposed system is practical and worth building. It checks the cost, the technology and the people who will use the system. The three main aspects studied for this project are given below.

3.1 ECONOMIC FEASIBILITY

The system is economically feasible because it needs no paid software. React, Node.js, Express, MySQL and Visual Studio Code are all free and open source. The system can run on the computers that the college already has, so no extra hardware has to be purchased. The time and paper saved by removing manual registers gives a further saving, so the benefit of the system is much greater than its cost.

3.2 TECHNICAL FEASIBILITY

The system uses widely available and well-supported technologies. A normal computer with a modern browser, Node.js and MySQL is enough to develop and run it. Node.js and MySQL can handle the number of records of a college without any difficulty, and the system can be moved to a web hosting server later if the college wants to use it over the internet. Therefore the project is technically feasible.

3.3 BEHAVIOURAL FEASIBILITY

The users of the system are the college admin, the staff and the students, and all of them are already familiar with web pages and forms. The screens are simple, and each user sees only the options required for his or her role. Since the system makes the daily work easier than the register method, the users are expected to accept it without much resistance. Only a short training is needed, so the system is behaviourally feasible.

CHAPTER 4

4. SYSTEM STUDY

4.1 EXISTING SYSTEM

In the existing system, the stock details of the college are written in registers or in separate spreadsheets kept by different departments. When a staff member or student needs an item, the request is made verbally or on paper, and the person in charge checks the register to see whether the item is available. The main drawbacks of this method are:

- Registers are difficult to search and update, and pages can be lost or damaged.
- Quantities are often wrong because the entries are made by hand at different times.
- There is no quick way to know which items are damaged or low in stock.
- Requests and approvals are not recorded properly.
- Preparing a stock report takes a lot of time and effort.

4.2 PROPOSED SYSTEM

The proposed system stores all stock details in a MySQL database and gives access to them through a web application. The admin manages the items and approves requests. Staff and students log in with their own accounts, view the available items and send requests. When the admin approves a request, the available quantity of that item is reduced automatically, so the stock figure is always correct. The advantages of the proposed system are:

- Stock details are stored in one place and can be searched instantly.
- The available quantity is updated automatically, which avoids counting errors.
- Each request and its result (approved, rejected or returned) is recorded.
- Low-stock and damaged items are highlighted for the admin.
- Role-based login keeps the data secure.

CHAPTER 5

5. TESTING AND IMPLEMENTATION

5.1 TESTING OBJECTIVE

Testing is the process of running a program with the aim of finding errors. A good test is one that has a high chance of finding an error that has not been found before. The objective of testing this project is to make sure that every page and every module works correctly, that wrong input is handled properly, and that the system produces the expected results.

5.2 TESTING PRINCIPLE

The following principles were followed while testing. All tests should be traceable to the requirements of the user. Testing should begin with small parts and move towards the whole system. Test cases should be planned before testing starts, and both correct and incorrect inputs should be tried. Testing should be repeated after every correction, because a change in one place may cause an error in another.

5.3 WHITE-BOX TESTING

White-box testing examines the internal logic of the program. The tester knows the code and prepares test cases so that every decision, loop and path is executed at least once. In this project, the conditions in the login check, the quantity check while requesting an item and the approval logic were tested in this way.

5.4 BLACK-BOX TESTING

Black-box testing checks the functions of the system without looking at the code. Inputs are given through the screens and the outputs are compared with the expected results. In this project the registration form, login form, add item form and request form were tested by giving valid, empty and wrong values.

5.5 TESTING STRATEGIES

A testing strategy joins the different test methods into a planned series of steps. For this project the strategy started with unit testing of each page, followed by integration testing of the pages together, system testing of the whole application and finally acceptance testing by the users.

5.6 TESTING FUNDAMENTALS

The fundamentals of testing are correctness, completeness and reliability. The system is correct when it gives the right output for a given input. It is complete when all the required functions are present, and it is reliable when it works properly every time it is used.

5.7 TESTING INFORMATION FLOW

The information flow in testing begins with the software configuration, which includes the requirements and the code, and the test configuration, which includes the test plan and test cases. The tests are run, the results are compared with the expected results, and any error found is corrected and tested again.

5.8 UNIT TESTING

In unit testing each module is tested separately. The registration page, login page, add item page, item list page, request page and approval page, together with the server routes behind them, were checked one by one to confirm that each of them works on its own.

5.9 INTEGRATION TESTING

Integration testing checks whether the modules work correctly when they are joined together. For example, an item added by the admin must appear in the request list of the student, and an approved request must reduce the available quantity shown on the admin dashboard.

5.10 SYSTEM TESTING

System testing checks the complete application in an environment like the real one. The whole flow of registering, logging in, adding stock, requesting an item, approving the request and viewing the updated stock was tested from beginning to end on different browsers.

5.11 ACCEPTANCE TESTING

Acceptance testing is done by the users to decide whether the system meets their needs. Sample staff and student accounts were created, and the users tried the system with real stock names. The suggestions received were noted and corrected before the final version was prepared.

5.12 TEST CASES

Some of the important test cases and their results are given in the table below.

S.No	Test case	Input	Expected output	Result
1	Register with a new email	Valid name, email, password	Registration successful message	Pass
2	Register with an existing email	Email already in database	Email already registered message	Pass
3	Login with correct details	Valid email and password	Redirect to the page of the role	Pass
4	Login with wrong password	Valid email, wrong password	Invalid email or password message	Pass
5	Open admin page as student	Student logged in	Access denied	Pass
6	Add an item	Chair, Furniture, 50	Item saved and shown in list	Pass

S.No	Test case	Input	Expected output	Result
7	Request more than available	Quantity above stock	Quantity not available message	Pass
8	Approve a request	Admin clicks Approve	Available quantity reduced	Pass
9	Reject a request	Admin clicks Reject	Status changes to Rejected	Pass
10	Low stock display	Available quantity below 5	Row highlighted in red	Pass

5.13 IMPLEMENTATION

Implementation is the stage where the tested system is installed and put into use. The database is created in MySQL, the Node.js server is started, the React front end is built and placed on the web server, the admin account is created, and the initial stock is entered. The users are then given their login details.

5.13.1 END USER TRAINING

A short training session is given to the admin, the staff and the students. It shows how to log in, how to search for items, how to send a request and, for the admin, how to add items and approve requests.

5.13.2 END USER EDUCATION

Users are told about the importance of keeping their passwords private, entering correct details and logging out after use. A simple help page or instruction sheet can be given for reference.

5.13.3 TRAINING OF APPLICATION SOFTWARE

The admin is trained to take a regular backup of the database, to create new categories and to manage user accounts. This makes sure that the system continues to work without depending on the developers.

5.13.4 POST IMPLEMENTATION VIEW

After the system is in use, its working is reviewed to find any problems and to collect suggestions for improvement. The feedback from the users is used to improve the screens and to add new features in future versions.

CHAPTER 6

6. SYSTEM CONFIGURATION

6.1 HARDWARE REQUIREMENTS

Component	Requirement
Processor	Intel Core i3 or higher
RAM	4 GB or more
Hard disk	500 GB (minimum 10 GB free space)
Monitor	14 inch colour monitor or larger
Keyboard and mouse	Standard keyboard and mouse

6.2 SOFTWARE REQUIREMENTS

Component	Requirement
Operating system	Windows 10 or higher
Front end	React 18 with React Router (JavaScript)
Back end	Node.js 18 or higher with Express
Database	MySQL 8
Build tool and packages	Vite and npm (Node Package Manager)
Browser	Google Chrome, Microsoft Edge or Mozilla Firefox
Code editor	Visual Studio Code

CHAPTER 7

7. COLLEGE ONLINE STOCK MANAGEMENT SYSTEM DATA FLOW DIAGRAM

A data flow diagram (DFD) shows how data moves through a system. It shows the external users, the processes, the data stores and the flow of data between them. The following diagrams describe the College Online Stock Management System at three levels.

7.1 0 LEVEL DFD FOR COLLEGE ONLINE STOCK MANAGEMENT SYSTEM

The level 0 DFD, also called the context diagram, shows the whole system as a single process. The admin, staff and students are the external entities that send data to the system and receive results from it, and the stock database is the data store used by the system.

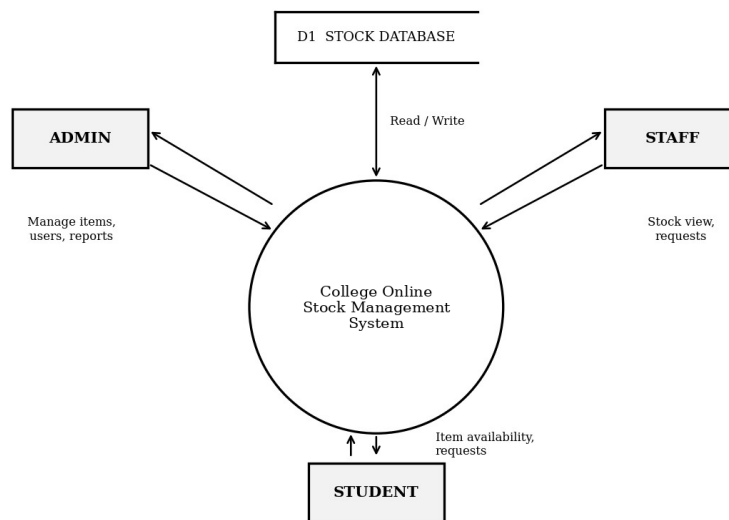


Figure 7.1 Level 0 DFD

7.2 LEVEL 1 DFD FOR COLLEGE ONLINE STOCK MANAGEMENT SYSTEM

The level 1 DFD divides the system into its main processes: login and registration, stock management, request management and damage reporting. Each process reads or writes the data stores for users, items, requests and damage records.

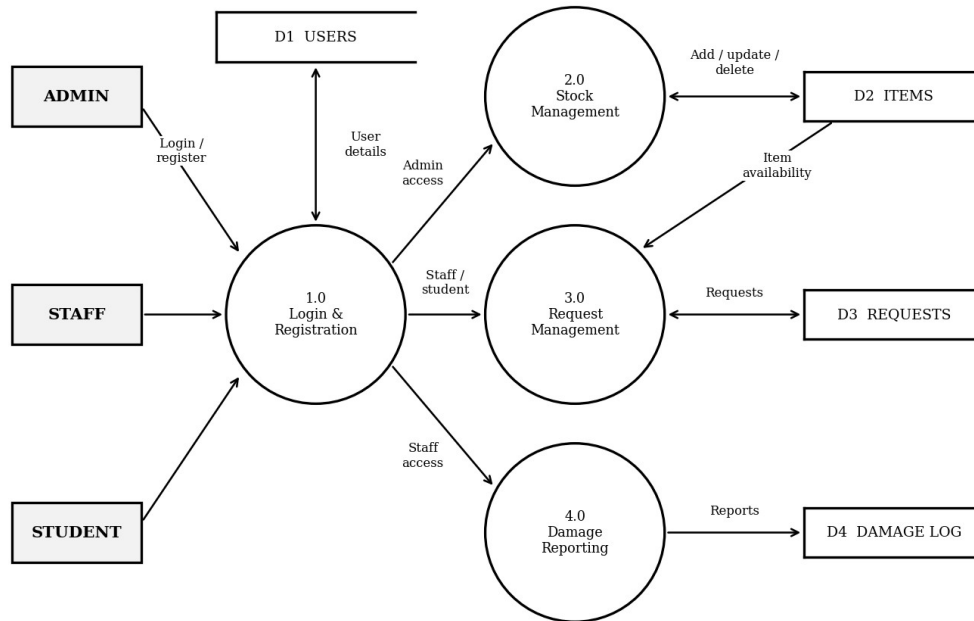


Figure 7.2 Level 1 DFD

7.3 LEVEL 2 DFD FOR COLLEGE ONLINE STOCK MANAGEMENT SYSTEM

The level 2 DFD explains the stock management process in detail. The admin can add an item, update an item, delete an item and view the stock. All these processes use the items data store.

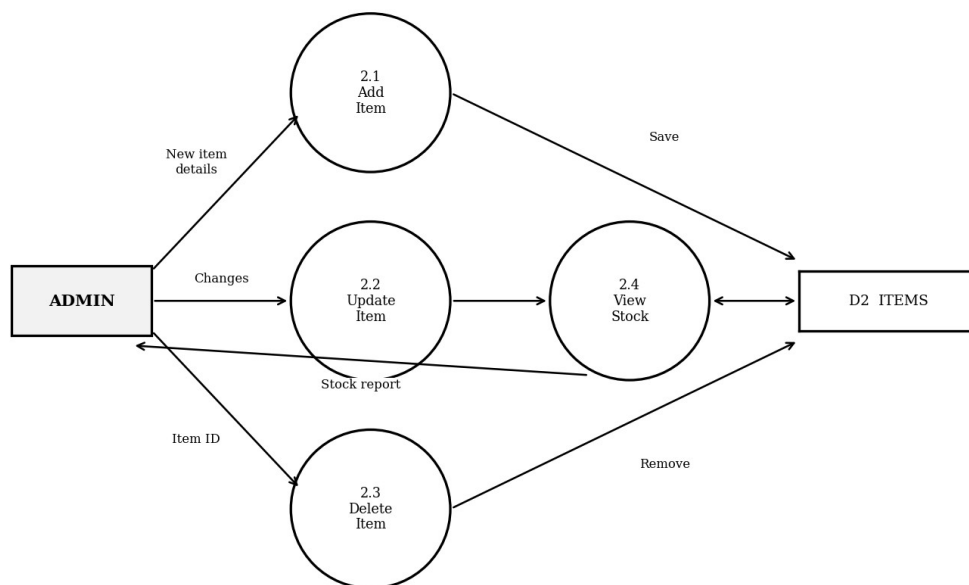


Figure 7.3 Level 2 DFD for stock management

CHAPTER 8

8. MODULES OF COLLEGE ONLINE STOCK MANAGEMENT SYSTEM

The system is divided into the following modules. Each module performs a separate job, and together they make up the complete application.

8.1 USER REGISTRATION

New staff members and students create an account by giving their name, email, password and role. The email must be unique, and the password is stored in encrypted form using the bcrypt hashing library of Node.js. The admin account is created directly in the database.

8.2 LOGIN MODULE

Registered users log in with their email and password. After the details are verified, the server sends a signed token (JSON Web Token) to the browser and the user is taken to the page for his or her role. If the details are wrong, an error message is shown. Every page and every server route checks the token, so an unregistered person cannot open the pages or the data by typing the address.

8.3 ADMIN MODULE

The admin is the main user of the system. The admin can view the dashboard with the total number of items, the total quantity and the available quantity. The admin can add new items, update or delete existing items, manage users, view the requests sent by staff and students, and approve or reject them. Items with low available quantity are highlighted.

8.4 STOCK MANAGEMENT MODULE

This module handles the records of chairs, tables, benches and laboratory equipment. For every item the name, category, total quantity, available quantity, location and condition are stored. The available quantity changes automatically whenever a request is approved or an item is returned.

8.5 STAFF MODULE

Staff members can log in, view the list of available items and send requests for the items they need for classes or laboratories. They can also report an item that is damaged or needs repair, so that the admin can take action.

8.6 STUDENT MODULE

Students can log in, view the available items and send requests for the items they need, for example lab equipment for a practical session. They can see whether their request is pending, approved or rejected.

CHAPTER 9

9. TECHNOLOGY USED

9.1 FRONT END TECHNOLOGY

React

React is an open-source JavaScript library for building user interfaces from small reusable pieces called components. A component describes a part of the screen using JSX, a tag-based syntax written inside JavaScript, and React updates only the parts of the page that change. In this project React is used to build the registration form, login form, item forms, the dashboard cards and the tables that display stock details. React Router is used to move between the pages without reloading the browser, and the `useState` and `useEffect` hooks are used to keep the form values and to load data from the server.

Styling in React

The look of the pages is defined with JavaScript style objects kept in one file (`styles.js`). This gives all pages a uniform look, arranges the summary cards on the dashboard and highlights low-stock rows in red.

Vite and npm

Vite is a build tool that creates and runs the React project quickly during development and prepares the final files for use. npm (Node Package Manager) is used to install the libraries needed by the project.

9.2 BACK END TECHNOLOGY

Node.js

Node.js is an open-source runtime that allows JavaScript to run on the server. It is fast because it handles many requests at the same time without waiting for each one to finish. In this project Node.js runs the server that receives requests from the React front end, checks the login details, verifies the user role and sends queries to the database.

Express

Express is a light web framework for Node.js. It is used to create the API routes of the project, such as `/api/auth/login`, `/api/items` and `/api/requests`, and to send the results back to the React pages in JSON format.

MySQL

MySQL is an open-source relational database management system. Data is stored in tables that are linked to each other by keys. This project uses tables for users, categories, items, requests and damage reports, and the tables are related through foreign keys. The mysql2 library is used to connect Node.js to MySQL, and parameterised queries (with ? placeholders) are used in all queries to prevent SQL injection.

bcrypt and JSON Web Token

The bcrypt library converts passwords into hashed form before they are stored, so the real password is never saved. After a successful login, the server creates a JSON Web Token (JWT) that contains the user id and role. The React front end sends this token with every request, and the server checks it before giving any data.

CHAPTER 10

10. CODING

10.1 PROJECT SETUP (commands)

```
# Back end (folder: server)
mkdir server && cd server
npm init -y
npm install express cors mysql2 bcryptjs jsonwebtoken
node index.js

# Front end (folder: client)
npm create vite@latest client -- --template react
cd client
npm install react-router-dom
npm run dev
```

10.2 DATABASE (database.sql)

```
CREATE DATABASE IF NOT EXISTS stock_management;
USE stock_management;

CREATE TABLE users (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(100) NOT NULL UNIQUE,
  password VARCHAR(255) NOT NULL,
  role ENUM('admin','staff','student') NOT NULL DEFAULT 'student',
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE categories (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(60) NOT NULL
);

CREATE TABLE items (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  category_id INT NOT NULL,
  total_qty INT NOT NULL DEFAULT 0,
  available_qty INT NOT NULL DEFAULT 0,
  location VARCHAR(100),
  condition_status ENUM('Good','Needs Repair','Damaged') DEFAULT 'Good',
  added_on TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (category_id) REFERENCES categories(id)
);

CREATE TABLE requests (
  id INT AUTO_INCREMENT PRIMARY KEY,
  user_id INT NOT NULL,
  item_id INT NOT NULL,
  qty INT NOT NULL,
  status ENUM('Pending','Approved','Rejected','Returned') DEFAULT 'Pending',
  requested_on TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (user_id) REFERENCES users(id),
  FOREIGN KEY (item_id) REFERENCES items(id)
);

CREATE TABLE damage_reports (
  id INT AUTO_INCREMENT PRIMARY KEY,
  item_id INT NOT NULL,
  user_id INT NOT NULL,
  description TEXT,
  reported_on TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (item_id) REFERENCES items(id),
  FOREIGN KEY (user_id) REFERENCES users(id)
);

INSERT INTO categories (name) VALUES ('Furniture'), ('Lab Equipment');
```

10.3 DATABASE CONNECTION (server/db.js)

```
// db.js - MySQL connection pool
const mysql = require("mysql2/promise");

const pool = mysql.createPool({
  host: "localhost",
  user: "root",
  password: "",
  database: "stock_management",
  waitForConnections: true,
  connectionLimit: 10,
});

module.exports = pool;
```

10.4 LOGIN CHECK (server/middleware/auth.js)

```
// auth.js - checks the token and the role of the user
const jwt = require("jsonwebtoken");

const SECRET = process.env.JWT_SECRET || "change-this-secret";

// requireLogin() -> any logged-in user
// requireLogin("admin") -> only the admin
function requireLogin(role) {
  return (req, res, next) => {
    const token = (req.headers.authorization || "").replace("Bearer ", "");
    try {
      req.user = jwt.verify(token, SECRET);
    } catch (err) {
      return res.status(401).json({ message: "Please log in." });
    }
    if (role && req.user.role !== role) {
      return res.status(403).json({ message: "Access denied." });
    }
    next();
  };
}

module.exports = { requireLogin, SECRET };
```

10.5 REGISTRATION AND LOGIN API (server/routes/auth.js)

```
const express = require("express");
const bcrypt = require("bcryptjs");
const jwt = require("jsonwebtoken");
const db = require("../db");
const { SECRET } = require("../middleware/auth");

const router = express.Router();

router.post("/register", async (req, res) => {
  const name = (req.body.name || "").trim();
  const email = (req.body.email || "").trim();
  const password = req.body.password || "";
  const role = ["staff", "student"].includes(req.body.role)
    ? req.body.role
    : "student";

  if (!name || !email || password.length < 6) {
    return res.status(400).json({ message: "Please fill all fields correctly." });
  }
  const hash = await bcrypt.hash(password, 10);
  try {
    await db.execute(
      "INSERT INTO users (name, email, password, role) VALUES (?, ?, ?, ?)",
      [name, email, hash, role]
    );
    res.json({ message: "Registration successful. You can log in now." });
  } catch (err) {
    if (err.code === "ER_DUP_ENTRY") {
      return res.status(409).json({ message: "This email is already registered." });
    }
    res.status(500).json({ message: "Registration failed." });
  }
});

router.post("/login", async (req, res) => {
  const email = (req.body.email || "").trim();
  const [rows] = await db.execute(
    "SELECT id, name, password, role FROM users WHERE email = ?",
    [email]
  );
  const user = rows[0];
  if (!user || !(await bcrypt.compare(req.body.password || "", user.password))) {
    return res.status(401).json({ message: "Invalid email or password." });
  }
  const token = jwt.sign(
    { id: user.id, name: user.name, role: user.role },
    SECRET,
    { expiresIn: "4h" }
  );
  res.json({ token, name: user.name, role: user.role });
});

module.exports = router;
```

10.6 STOCK API (server/routes/items.js)

```
const express = require("express");
const db = require("../db");
const { requireLogin } = require("../middleware/auth");

const router = express.Router();

// List of all items (any logged-in user)
router.get("/", requireLogin(), async (req, res) => {
  const [rows] = await db.execute(
    `SELECT i.id, i.name, c.name AS category, i.total_qty, i.available_qty,
      i.location, i.condition_status
      FROM items i JOIN categories c ON i.category_id = c.id
      ORDER BY c.name, i.name`
  );
  res.json(rows);
});

// Dashboard totals (admin only)
router.get("/summary", requireLogin("admin"), async (req, res) => {
  const [[totals]] = await db.execute(
    `SELECT COUNT(*) AS kinds, IFNULL(SUM(total_qty),0) AS total,
      IFNULL(SUM(available_qty),0) AS available FROM items`
  );
  const [[pending]] = await db.execute(
    "SELECT COUNT(*) AS n FROM requests WHERE status='Pending'"
  );
  res.json({ ...totals, pending: pending.n });
});

router.get("/categories", requireLogin("admin"), async (req, res) => {
  const [rows] = await db.execute("SELECT id, name FROM categories");
  res.json(rows);
});

// Add a new item (admin only)
router.post("/", requireLogin("admin"), async (req, res) => {
  const name = (req.body.name || "").trim();
  const cat = parseInt(req.body.category_id, 10);
  const qty = Math.max(0, parseInt(req.body.qty, 10) || 0);
  const loc = (req.body.location || "").trim();

  if (!name || !cat) {
    return res.status(400).json({ message: "Could not add item." });
  }
  try {
    await db.execute(
      `INSERT INTO items (name, category_id, total_qty, available_qty, location)
      VALUES (?, ?, ?, ?, ?)`,
      [name, cat, qty, qty, loc]
    );
    res.json({ message: "Item added successfully." });
  } catch (err) {
    res.status(500).json({ message: "Could not add item." });
  }
});

module.exports = router;
```

10.7 REQUEST API (server/routes/requests.js)

```
const express = require("express");
const db = require("../db");
const { requireLogin } = require("../middleware/auth");

const router = express.Router();

// Staff / student sends a request
router.post("/", requireLogin(), async (req, res) => {
  const item = parseInt(req.body.item_id, 10);
  const qty = parseInt(req.body.qty, 10);

  const [rows] = await db.execute(
    "SELECT available_qty FROM items WHERE id = ?", [item]
  );
  if (!rows[0] || !(qty >= 1) || qty > rows[0].available_qty) {
    return res.status(400).json({ message: "Requested quantity is not available." });
  }
  await db.execute(
    "INSERT INTO requests (user_id, item_id, qty) VALUES (?, ?, ?)",
    [req.user.id, item, qty]
  );
  res.json({ message: "Request sent to the admin." });
});

// Admin sees all requests
router.get("/", requireLogin("admin"), async (req, res) => {
  const [rows] = await db.execute(
    `SELECT r.id, u.name AS user, u.role, i.name AS item, r.qty, r.status
    FROM requests r
    JOIN users u ON r.user_id = u.id
    JOIN items i ON r.item_id = i.id
    ORDER BY r.requested_on DESC`
  );
  res.json(rows);
});

// Admin approves or rejects a request
router.put("/:id/:action", requireLogin("admin"), async (req, res) => {
  const id = parseInt(req.params.id, 10);
  const action = req.params.action;

  const [rows] = await db.execute(
    "SELECT item_id, qty, status FROM requests WHERE id = ?", [id]
  );
  const request = rows[0];
  if (!request || request.status !== "Pending") {
    return res.status(400).json({ message: "Request already processed." });
  }

  if (action === "reject") {
    await db.execute("UPDATE requests SET status='Rejected' WHERE id = ?", [id]);
    return res.json({ message: "Request rejected." });
  }
  if (action !== "approve") {
    return res.status(400).json({ message: "Unknown action." });
  }

  const conn = await db.getConnection();
  try {
    await conn.beginTransaction();
    const [result] = await conn.execute(
      `UPDATE items SET available_qty = available_qty - ?
      WHERE id = ? AND available_qty >= ?`,
      [request.qty, request.item_id, request.qty]
    );
    if (result.affectedRows === 1) {
      await conn.execute("UPDATE requests SET status='Approved' WHERE id = ?", [id]);
      await conn.commit();
      res.json({ message: "Request approved." });
    } else {

```

```

        await conn.rollback();
        res.status(400).json({ message: "Not enough quantity available." });
    }
} catch (err) {
    await conn.rollback();
    res.status(500).json({ message: "Could not approve the request." });
} finally {
    conn.release();
}
});

module.exports = router;

```

10.8 SERVER START (server/index.js)

```

const express = require("express");
const cors = require("cors");

const app = express();
app.use(cors());
app.use(express.json());

app.use("/api/auth", require("./routes/auth"));
app.use("/api/items", require("./routes/items"));
app.use("/api/requests", require("./routes/requests"));

app.listen(5000, () => {
    console.log("Server running on http://localhost:5000");
});

```

10.9 API HELPER AND STYLES (client/src/api.js, styles.js)

```
// api.js - sends requests to the Node.js server with the login token
const BASE = "http://localhost:5000/api";

export async function api(path, method = "GET", body) {
  const token = localStorage.getItem("token");
  const res = await fetch(BASE + path, {
    method,
    headers: {
      "Content-Type": "application/json",
      ...(token ? { Authorization: "Bearer " + token } : {}),
    },
    body: body ? JSON.stringify(body) : undefined,
  });
  const data = await res.json();
  return { ok: res.ok, data };
}

// styles.js - style objects used by all pages
export const s = {
  page: { fontFamily: "Arial, sans-serif", background: "#eef2f7",
    minHeight: "100vh", padding: 30, boxSizing: "border-box" },
  box: { background: "#fff", maxWidth: 380, margin: "0 auto", padding: 24,
    borderRadius: 8, boxShadow: "0 2px 8px rgba(0,0,0,0.15)" },
  wide: { background: "#fff", maxWidth: 950, margin: "0 auto", padding: 24,
    borderRadius: 8, boxShadow: "0 2px 8px rgba(0,0,0,0.15)" },
  title: { textAlign: "center", color: "#1f3a5f" },
  input: { display: "block", width: "100%", padding: 10, margin: "10px 0",
    boxSizing: "border-box" },
  button: { display: "block", width: "100%", padding: 10, margin: "10px 0",
    boxSizing: "border-box", background: "#1f3a5f", color: "#fff",
    border: "none", cursor: "pointer" },
  msg: { color: "#b00020", textAlign: "center" },
  link: { color: "#1f3a5f", cursor: "pointer", textDecoration: "underline" },
  table: { width: "100%", borderCollapse: "collapse", marginTop: 15 },
  th: { border: "1px solid #ccc", padding: 8, textAlign: "left",
    background: "#1f3a5f", color: "#fff" },
  td: { border: "1px solid #ccc", padding: 8, textAlign: "left" },
  low: { background: "#ffe5e5" },
  cards: { display: "flex", gap: 15, margin: "15px 0" },
  card: { flex: 1, background: "#f5f8fc", padding: 12, textAlign: "center",
    borderRadius: 6 },
};
```

10.10 APP AND ROUTES (client/src/main.jsx, App.jsx)

```
// main.jsx
import React from "react";
import ReactDOM from "react-dom/client";
import { BrowserRouter } from "react-router-dom";
import App from "./App";

ReactDOM.createRoot(document.getElementById("root")).render(
  <BrowserRouter>
    <App />
  </BrowserRouter>
);

// App.jsx
import { Routes, Route, Navigate } from "react-router-dom";
import Register from "./Register";
import Login from "./Login";
import Dashboard from "./Dashboard";
import AddItem from "./AddItem";
import RequestItem from "./RequestItem";
import ApproveRequests from "./ApproveRequests";

// Opens a page only for logged-in users of the correct role
function Protected({ role, children }) {
  if (!localStorage.getItem("token")) return <Navigate to="/login" />;
  if (role && localStorage.getItem("role") !== role) {
    return <p style={{ padding: 30 }}>Access denied.</p>;
  }
  return children;
}

export default function App() {
  return (
    <Routes>
      <Route path="/" element={<Navigate to="/login" />} />
      <Route path="/register" element={<Register />} />
      <Route path="/login" element={<Login />} />
      <Route path="/dashboard"
        element={<Protected role="admin"><Dashboard /></Protected>} />
      <Route path="/add-item"
        element={<Protected role="admin"><AddItem /></Protected>} />
      <Route path="/requests"
        element={<Protected role="admin"><ApproveRequests /></Protected>} />
      <Route path="/request-item"
        element={<Protected><RequestItem /></Protected>} />
    </Routes>
  );
}
```

10.11 REGISTRATION PAGE (client/src/Register.jsx)

```
import { useState } from "react";
import { Link } from "react-router-dom";
import { api } from "../api";
import { s } from "../styles";

export default function Register() {
  const [form, setForm] = useState({
    name: "", email: "", password: "", role: "student",
  });
  const [msg, setMsg] = useState("");

  const change = (e) => setForm({ ...form, [e.target.name]: e.target.value });

  const submit = async (e) => {
    e.preventDefault();
    const { data } = await api("/auth/register", "POST", form);
    setMsg(data.message);
  };

  return (
    <div style={s.page}>
      <div style={s.box}>
        <h2 style={s.title}>Create Account</h2>
        <p style={s.msg}>{msg}</p>
        <form onSubmit={submit}>
          <input style={s.input} name="name" placeholder="Full name"
            onChange={change} required />
          <input style={s.input} type="email" name="email" placeholder="Email"
            onChange={change} required />
          <input style={s.input} type="password" name="password"
            placeholder="Password" minLength={6} onChange={change} required />
          <select style={s.input} name="role" onChange={change}>
            <option value="student">Student</option>
            <option value="staff">Staff</option>
          </select>
          <button style={s.button} type="submit">Register</button>
        </form>
        <Link to="/login">Already registered? Login</Link>
      </div>
    </div>
  );
}
```

10.12 LOGIN PAGE (client/src/Login.jsx)

```
import { useState } from "react";
import { Link, useNavigate } from "react-router-dom";
import { api } from "../api";
import { s } from "../styles";

export default function Login() {
  const nav = useNavigate();
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [error, setError] = useState("");

  const submit = async (e) => {
    e.preventDefault();
    const { ok, data } = await api("/auth/login", "POST", { email, password });
    if (!ok) return setError(data.message);
    localStorage.setItem("token", data.token);
    localStorage.setItem("name", data.name);
    localStorage.setItem("role", data.role);
    nav(data.role === "admin" ? "/dashboard" : "/request-item");
  };

  return (
    <div style={s.page}>
      <div style={s.box}>
        <h2 style={s.title}>College Online Stock Management System</h2>
        <p style={s.msg}>{error}</p>
        <form onSubmit={submit}>
          <input style={s.input} type="email" placeholder="Email"
            onChange={(e) => setEmail(e.target.value)} required />
          <input style={s.input} type="password" placeholder="Password"
            onChange={(e) => setPassword(e.target.value)} required />
          <button style={s.button} type="submit">Login</button>
        </form>
        <Link to="/register">New user? Register</Link>
      </div>
    </div>
  );
}
```

10.13 ADMIN DASHBOARD (client/src/Dashboard.jsx)

```
import { useEffect, useState } from "react";
import { Link, useNavigate } from "react-router-dom";
import { api } from "../api";
import { s } from "../styles";

export default function Dashboard() {
  const nav = useNavigate();
  const [items, setItems] = useState([]);
  const [sum, setSum] = useState({ kinds: 0, total: 0, available: 0, pending: 0 });

  useEffect(() => {
    api("/items").then((r) => setItems(r.data));
    api("/items/summary").then((r) => setSum(r.data));
  }, []);

  const logout = () => {
    localStorage.clear();
    nav("/login");
  };

  // rows with available quantity below 5 are highlighted in red
  const cell = (low) => (low ? { ...s.td, ...s.low } : s.td);

  return (
    <div style={s.page}>
      <div style={s.wide}>
        <h2 style={s.title}>Admin Dashboard</h2>
        <p>
          Welcome, {localStorage.getItem("name")} |{" "}
          <Link to="/add-item">Add Item</Link> |{" "}
          <Link to="/requests">Requests ({sum.pending})</Link> |{" "}
          <span style={s.link} onClick={logout}>Logout</span>
        </p>
        <div style={s.cards}>
          <div style={s.card}>Item types<br /><b>{sum.kinds}</b></div>
          <div style={s.card}>Total quantity<br /><b>{sum.total}</b></div>
          <div style={s.card}>Available<br /><b>{sum.available}</b></div>
        </div>
        <table style={s.table}>
          <thead>
            <tr>
              {["Item", "Category", "Total", "Available", "Location", "Condition"]}
                .map((h) => <th key={h} style={s.th}>{h}</th>)}
            </tr>
          </thead>
          <tbody>
            {items.map((row) => {
              const low = row.available_qty < 5;
              return (
                <tr key={row.id}>
                  <td style={cell(low)}>{row.name}</td>
                  <td style={cell(low)}>{row.category}</td>
                  <td style={cell(low)}>{row.total_qty}</td>
                  <td style={cell(low)}>{row.available_qty}</td>
                  <td style={cell(low)}>{row.location}</td>
                  <td style={cell(low)}>{row.condition_status}</td>
                </tr>
              );
            })}
          </tbody>
        </table>
      </div>
    </div>
  );
}
```

10.14 ADD ITEM PAGE (client/src/AddItem.jsx)

```
import { useEffect, useState } from "react";
import { Link } from "react-router-dom";
import { api } from "../api";
import { s } from "../styles";

export default function AddItem() {
  const [cats, setCats] = useState([]);
  const [form, setForm] = useState({
    name: "", category_id: "", qty: "", location: "",
  });
  const [msg, setMsg] = useState("");

  useEffect(() => {
    api("/items/categories").then((r) => {
      setCats(r.data);
      if (r.data.length) setForm((f) => ({ ...f, category_id: r.data[0].id }));
    });
  }, []);

  const change = (e) => setForm({ ...form, [e.target.name]: e.target.value });

  const submit = async (e) => {
    e.preventDefault();
    const { data } = await api("/items", "POST", form);
    setMsg(data.message);
  };

  return (
    <div style={s.page}>
      <div style={s.box}>
        <h2 style={s.title}>Add Item</h2>
        <p style={s.msg}>{msg}</p>
        <form onSubmit={submit}>
          <input style={s.input} name="name" onChange={change} required
            placeholder="Item name (e.g. Chair, Microscope)" />
          <select style={s.input} name="category_id" onChange={change}>
            {cats.map((c) => (
              <option key={c.id} value={c.id}>{c.name}</option>
            ))}
          </select>
          <input style={s.input} type="number" name="qty" min="0"
            placeholder="Quantity" onChange={change} required />
          <input style={s.input} name="location" onChange={change}
            placeholder="Location (e.g. Physics Lab)" />
          <button style={s.button} type="submit">Save Item</button>
        </form>
        <Link to="/dashboard">Back to dashboard</Link>
      </div>
    </div>
  );
}
```

10.15 REQUEST ITEM PAGE (client/src/RequestItem.jsx)

```
import { useEffect, useState } from "react";
import { useNavigate } from "react-router-dom";
import { api } from "../api";
import { s } from "../styles";

export default function RequestItem() {
  const nav = useNavigate();
  const [items, setItems] = useState([]);
  const [itemId, setItemId] = useState("");
  const [qty, setQty] = useState("");
  const [msg, setMsg] = useState("");

  const load = () => {
    api("/items").then((r) => {
      const available = r.data.filter((i) => i.available_qty > 0);
      setItems(available);
      if (available.length) setItemId(available[0].id);
    });
  };
  useEffect(() => { load(); }, []);

  const submit = async (e) => {
    e.preventDefault();
    const { data } = await api("/requests", "POST", { item_id: itemId, qty });
    setMsg(data.message);
  };

  const logout = () => {
    localStorage.clear();
    nav("/login");
  };

  return (
    <div style={s.page}>
      <div style={s.box}>
        <h2 style={s.title}>Request Item</h2>
        <p>Welcome, {localStorage.getItem("name")}</p>
        <p style={s.msg}>{msg}</p>
        <form onSubmit={submit}>
          <select style={s.input} onChange={(e) => setItemId(e.target.value)}>
            {items.map((i) => (
              <option key={i.id} value={i.id}>
                {i.name} (available: {i.available_qty})
              </option>
            ))}
          </select>
          <input style={s.input} type="number" min="1" placeholder="Quantity"
            onChange={(e) => setQty(e.target.value)} required />
          <button style={s.button} type="submit">Send Request</button>
        </form>
        <span style={s.link} onClick={logout}>Logout</span>
      </div>
    </div>
  );
}
```

10.16 APPROVE REQUEST PAGE (client/src/ApproveRequests.jsx)

```
import { useEffect, useState } from "react";
import { Link } from "react-router-dom";
import { api } from "../api";
import { s } from "../styles";

export default function ApproveRequests() {
  const [list, setList] = useState([]);

  const load = () => api("/requests").then((r) => setList(r.data));
  useEffect(() => { load(); }, []);

  // action is "approve" or "reject"
  const act = async (id, action) => {
    await api(`/requests/${id}/${action}`, "PUT");
    load();
  };

  return (
    <div style={s.page}>
      <div style={s.wide}>
        <h2 style={s.title}>Item Requests</h2>
        <Link to="/dashboard">Back to dashboard</Link>
        <table style={s.table}>
          <thead>
            <tr>
              <th>["User", "Role", "Item", "Qty", "Status", "Action"]
                .map((h) => <th key={h} style={s.th}>{h}</th>)}
            </tr>
          </thead>
          <tbody>
            {list.map((r) => (
              <tr key={r.id}>
                <td style={s.td}>{r.user}</td>
                <td style={s.td}>{r.role}</td>
                <td style={s.td}>{r.item}</td>
                <td style={s.td}>{r.qty}</td>
                <td style={s.td}>{r.status}</td>
                <td style={s.td}>
                  {r.status === "Pending" && (
                    <>
                      <span style={s.link} onClick={() => act(r.id, "approve")}>
                        Approve
                      </span>
                      {" | "}
                      <span style={s.link} onClick={() => act(r.id, "reject")}>
                        Reject
                      </span>
                    </>
                  )}
                </td>
              </tr>
            ))}
          </tbody>
        </table>
      </div>
    </div>
  );
}
```

CHAPTER 11

11. SCREEN SHOTS

The following figures show the main screens of the College Online Stock Management System.

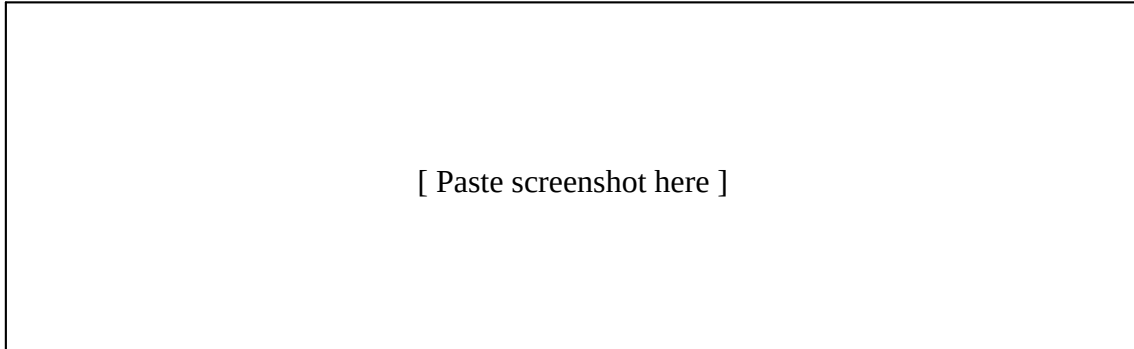


Figure 11.1 Registration page

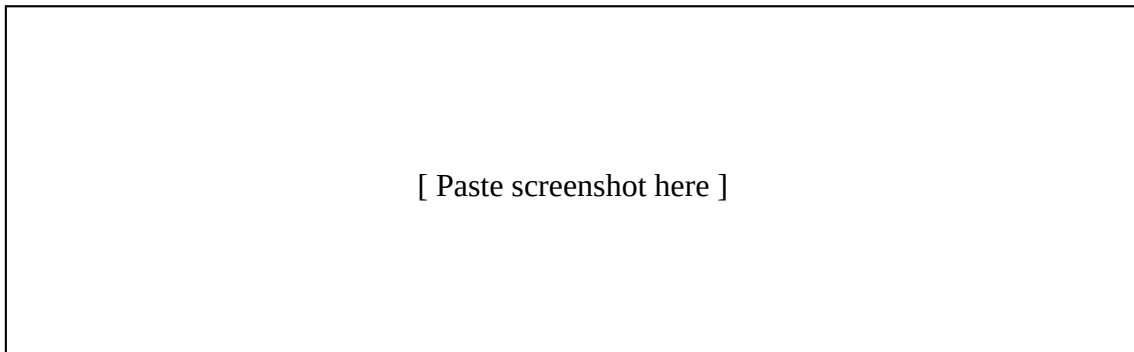


Figure 11.2 Login page

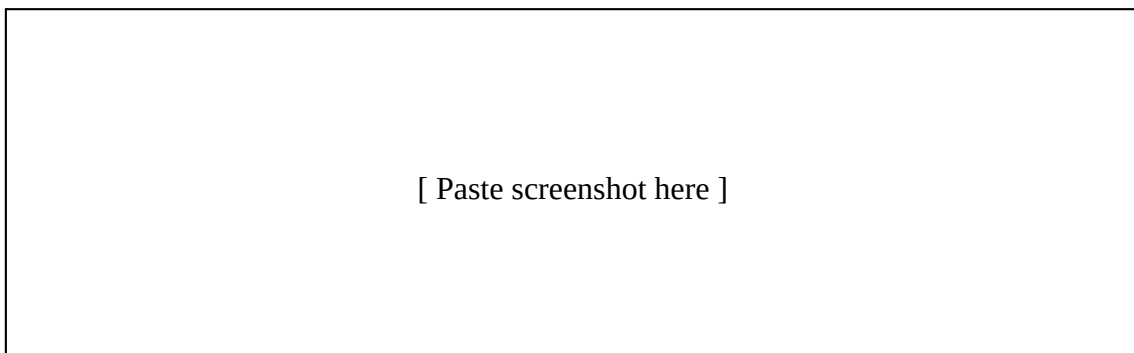


Figure 11.3 Admin dashboard

[Paste screenshot here]

Figure 11.4 Add item page

[Paste screenshot here]

Figure 11.5 Request item page (staff / student)

[Paste screenshot here]

Figure 11.6 Approve request page

CHAPTER 12

12. CONCLUSION

The College Online Stock Management System replaces the manual stock registers of the college with a simple and secure web application. It keeps accurate records of furniture and laboratory equipment, allows staff and students to request items online, and updates the available quantity automatically when the admin approves a request. The system saves time and paper, avoids counting errors and gives the management clear and current information about the stock.

The project was developed using React, Node.js and MySQL, and it was tested with different kinds of input to make sure that it works correctly. In the future the system can be improved by adding barcode or QR code scanning, email and SMS alerts for low stock, purchase and vendor records, downloadable PDF reports, and a mobile application. With these additions it can be used by several departments or campuses at the same time.

CHAPTER 13

13. REFERENCE

BOOKS

- Roger S. Pressman, Software Engineering: A Practitioner's Approach, McGraw-Hill Education.
- Ramez Elmasri and Shamkant B. Navathe, Fundamentals of Database Systems, Pearson.
- Alex Banks and Eve Porcello, Learning React, O'Reilly Media.
- Ethan Brown, Web Development with Node and Express, O'Reilly Media.

WEBSITES

- <https://react.dev>
- <https://nodejs.org>